2023 IEEE 22nd International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom) | 979-8-3503-8199-3/23/$31.00 ©2023 IEEE | DOI: 10.1109/TrustCom60117.2023.00024

Representation-enhanced APT Detection Using Contrastive Learning

Fengxi Zhou
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
zhoufengxi@iie.ac.cn

Baoming Chang
Institute of Information Engineering, Chinese Academy of Sciences
School of Cyber Security, University of Chinese Academy of Sciences
Beijing, China
changbaoming@iie.ac.cn

Yu Wen
Institute of Information Engineering, Chinese Academy of Sciences
Beijing, China
wenyu@iie.ac.an

Dan Meng
Institute of Information Engineering, Chinese Academy of Sciences
Beijing, China
mengdan@iie.ac.cn

***Abstract*—Advanced Persistent Threats (APTs) are low-and-slow attack patterns and difficult to detect due to their strong concealment. Recently, provenance-based method demonstrates promising prospect in APT detection. However, existing approaches suffer from following several limitations. First, rule-based models heavily rely on domain-specific knowledges and sophisticated matching mechanism from rules to system logs. Second, current anomaly-based techniques require attack-free data to train model and hard to acquire fine-grained detection results to locate attack events. An important reason for undermining the detection capacity of model is the notorious dependency explosion problem and data imbalance problem. Both of problems make the vector representations of benign events and attack events similar, therefore, the model can not differentiate them well. In this paper, we propose DEEPDIST, which can enhance the entity vector representation by contrastive learning. In this way, we force the projection of benign and attack entities into different feature regions to improve the detection ability of the model. Then we correlate these detected entities to constitute attack events. During our evaluation against thirteen real APT attack scenarios of two datasets, DEEPDIST shows the detection results with high accuracy.**

***Index Terms*—APT detection, provenance graph, contrastive learning**

I. INTRODUCTION

In recent years, cyberspace security threats are changing dramatically, with stronger purpose, attackers may adopt more covert tactics to bypass conventional defense means (e.g., firewall). Among these advanced strategies are Advanced Persistent Threats (APTs), which are more sophisticated and more stealthy than traditional attacks and involve multiple stages [1]. Exploiting the exploited vulnerabilities allows an attacker to remain undetected in a target host for a long time. During this period, the attacker can steal sensitive data or conducting further malicious activities continuously [2]. For example, during the investigation into Tomiris's activities in 2022, Kaspersky identified the use of a previously undocumented implant developed in Rust, dubbed "JLORAT", which was in operation as early as August 2022 and remained active into 2023 [3]. Detecting and defending against APT attacks presents a formidable challenge due to their high variability and concealment. It is therefore essential to implement effective measures for detecting APT attacks promptly.

At present, The use of provenance graph based detection [4–8] has gained traction as a promising approach to combat APT attacks. This method uses system logs that come from system monitor to construct provenance graph in a form of directed acyclic graph (DAG) where the nodes indicate system entities like processes, files and network connections, while the edges represent specific events within the system [9]. Although APT attacks may evade conventional intrusion detection tools, they are still inevitably to leave footprints in system logs [10–12]. By leveraging the provenance graph which provides the rich contextual information of events, APT detection tools can correlate events and their contextual information to recover attack campaign [13, 14]. Unfortunately, existing approaches suffer from several major limitations. First, despite the efficiency and state-of-the-art performance achieved by rule-based APT detection techniques, these methods require domain-specific knowledge and are difficult to expand to different system environment. For example, to perform rule matching, these methods necessitate knowledge of the file classification level [15]. Nevertheless, the file classification level is different in different systems, making it difficult to extend the designed rules to different system environment. Second, due to the problems of dependence explosion [16–18] and data imbalance, only a few malicious events are submerged in a large number of benign events, making it difficult for learning-based solutions to determine specific attack locations, resulting in coarse-grained detection results [4, 10] or high false positive rates [19]. Moreover, these learning-based techniques demand attack-free data to train model, which is impractical in real-

2324-9013/23/$31.00 ©2023 IEEE
DOI 10.1109/TrustCom60117.2023.00024

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 27,2026 at 16:25:56 UTC from IEEE Xplore. Restrictions apply.

life implement scenario because it is hard to guarantee that real-world audit records are perfectly clean.

To address the aforementioned issues, we propose DEEPDIST, which aims to achieve automated fine-grained detection even in the absence of attack-free data. However, there are several major challenges that must be solved.

Challenge 1: Distinct entity representation for attack entity and benign entity. When an entity participates in multiple events, and one of those events is malicious, it's labeled as a malicious entity. This makes it challenging to distinguish between entities with only a few malicious events among numerous benign ones and truly benign entities.

Challenge 2: How to model an entity within a proper scope? In the field of anomaly detection in graphs, it is typical to use the n-order neighbors [10, 19] of a node to model it for subsequent representation. However, if n is too large, benign entities may include malicious entities causing the detection model to degrade into coarse-grained detection. If n is too small, it is difficult to model an entity's attack behavior because APT attacks usually involve an attack chain. Therefore, finding a suitable range for entity representation pose a challenge.

Challenge 3: Data imbalance. In scenarios for detecting APT attacks, the number of attacking entities and non-attacking entities is extremely imbalanced, with typically less than 1% being attacking entities. This highly unbalanced data is very detrimental to model training. This data imbalance poses a significant challenge for model training. Although simple oversampling and undersampling methods can solve the problem caused by imbalanced data, these methods may lead to overfitting or information loss in the model [13]. Thus, accurately modeling the intricate and nuanced relationship between attacking and non-attacking entities when dealing with imbalanced data constitutes a challenge to enhance both performance and generalization of models.

For Challenge 1, our DEEPDIST need to project the representation vectors of attack entities and benign entities to obviously different spatial regions. In the optimal situation, the representation vectors of attack entities are mapped into one region while the representation vectors of benign entities are mapped into another absolutely different region. In other words, the ideal situation is to attain a binary classification of benign entities and attacking entities. However, it is not easy because of the reasons above. Fortunately, recent contrastive learning [20–22] makes huge progress in vector representations in computer vision which can increase the distinction between representations of different classes. Inspired by this, DEEPDIST uses a contrastive learning scheme to strengthen the representation of vectors. For Challenge 2, we abandoned the approach of representing nodes with excessively high n-order neighbors, because that would introduce attack entities for benign nodes. Instead, we consider only the first-order neighbors of the entity. At the same time, in order to make up for the insufficient representation of entity information by first-order neighbors, we additionally introduce a chain obtained by depth-first search for each entity. For challenge 3, we propose a customized data augmentation scheme that does not change the ground truth that indicates whether an entity is malicious or benign.

We implement DEEPDIST in real-world APT attack dataset released by ATLAS [13] and our evaluation results show that DEEPDIST can complete APT detection with high accuracy. Specifically, DEEPDIST achieves 94.91% precision, 94.47% recall and 94.67% F1-score in ten APT attack scenarios averagely. We also applied DEEPDIST on our own simulation dataset and achieved 100% accuracy.

Contributions:

- We present DEEPDIST, a novel approach for automated fine-grained detection of Advanced Persistent Threat (APT) attacks, even in the absence of attack-free data.
- We offer a novel and tailored approach to entity modeling that strikes a balance between fine-grained detection and detection accuracy.
- We first apply contrastive learning to the ATP detection field, which helps to clarify the vector representation of attacking and malicious entities.
- We propose a customized entity data augmentation scheme that does not change the ground truth that indicates whether an entity is attack or benign.

II. BACKGROUND AND MOTIVATION

A. System Log and Provenance Graph

In this paper, the Provenance graph refers to System-level Provenance Graph which parses the system audit logs to a directed acyclic graph (DAG). Generally, each entry in these system logs can be represented as an event, and each event can be expressed as a quartet $< sub, rel, obj, t >$. In accordance with the previous works [14, 23], the sub refers to subject which is a process in system and the obj refers to processes, files or networks. The rel refers to the relationship between a subject and object, which can take various forms, such as between processes and processes (e.g., fork), processes and files (e.g., read and write), or processes and networks (e.g., recvmsg). The t indicates the timestamp that indicates when the event occurred. In the provenance graph, the nodes represent the subjects and objects while the edges are the relations between them.

B. Contrastive Learning

Contrastive learning [20, 21] is a machine learning method that learns representations of samples by comparing positive pairs of similar samples and negative pairs of dissimilar samples. In contrastive learning, we project similar samples to nearby positions in an embedding space and dissimilar samples to distant positions, so that we can learn more discriminative representations of the samples for downstream tasks such as classification, retrieval, and clustering. Contrastive learning typically requires a large amount of data and computational resources for training. In recent research, deep neural networks have been successfully used for learning contrastive representations, such as the SimCLR [20], MoCo [21] and BYOL [22]

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 27,2026 at 16:25:56 UTC from IEEE Xplore. Restrictions apply.

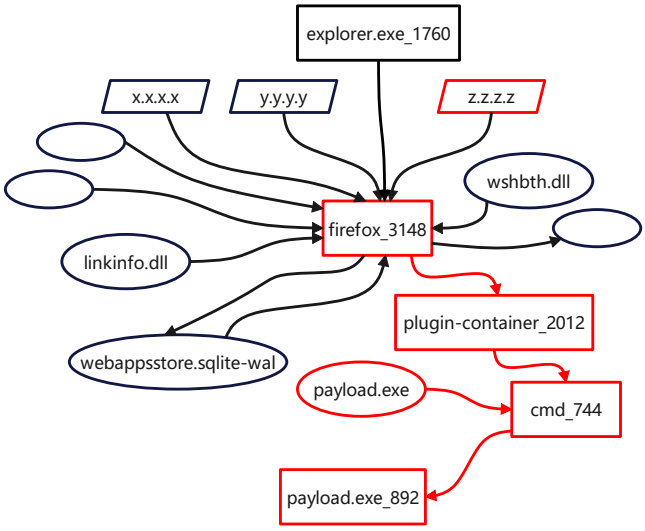


Fig. 1. Example of motivation.

algorithms. These algorithms employ strategies such as large-scale data augmentation and sampling from multiple views to improve the effectiveness of contrastive learning.

C. Motivation

Within our paper, Fig. 1 illustrates an example of motivation. This figure shows an attacker exploiting the `CVE-2015-5122` [24] vulnerability to insert malicious content into Flash, thereby tricking users into visiting a malicious website to execute malicious code. The diagram uses rectangles to represent processes, ellipses to represent files, and parallelograms to represent network connections. The components highlighted in red denote the events associated with the attack.

It can be clearly seen from the figure that the firefox process is a compromised entity, however, it is connected to only two malicious neighbors (`z.z.z.z` and `plugin-container_2012`) while having multiple neighbor nodes that are not associated with the attack. The existing learning-based detection approaches such as Unicorn [10] and StreamSpot [4], are graph-level detection. In essence, their methods use the n-order neighbors of a node to represent an entity, where benign processes (such as `explorer.exe_1760`) can be falsely implicated in attacks (such as `firefox_3148`), leading to coarse-grained detection results. Moreover, these methods require attack-free data to train model. The DEEPDIST proposed in this paper, which can emphasize the attack events (the red part) for the representation of the firefox process and deemphasize other events by contrastive learning. Moreover, DEEPDIST is designed to consider two aspects for representing firefox: the one-order neighbor and one path (e.g., `firefox_3148`→`plugin-container_2012`→`cmd_744`→`payload.exe_892`).

III. PROBLEM DEFINITION

A. Problem Statement

Given a provenance graph G built by system logs, our goal is to find the attack process entities in the graph without labeled data or attack-free data to train beforehand. Then, we can obtain a provenance graph G that contains labeled processes and use these labeled processes to pinpoint the APT attack campaign.

B. Threat Model

In our threat model, we consider a scenario where an attacker gains access to the system through information technology tactics, such as phishing emails, and seeks to obtain sensitive information from within the organization. We assume that the system's underlying OS and security measures are part of the trusted computing base (TCB) and are secure from tampering. Our focus is on detecting and mitigating attacks that exploit vulnerabilities in the system software or third-party applications. We do not consider attacks that exploit hardware vulnerabilities or physical access to the system. Our goal is to identify the attack methods used by the attacker, such as remote code execution, privilege escalation, and lateral movement to other hosts, and prevent exfiltration of sensitive data. We further assume that provenance graphs remain intact and secure, and are not susceptible to tampering by malicious individuals.

IV. DESIGN

In this section, we first provide an outline of DEEPDIST's structure and then we will present each major component in detail.

A. Overview

Fig. 2 illustrates the architecture of DEEPDIST, which utilizes the contrastive learning to achieve the APT detection task by node classification. Intuitively, our approach is to find the similarity between entities of the same type. Once finding the similarity, the model will project these entities to nearby location in vector space. The design of DEEPDIST mainly consists of following several modules: ❶ Provenance Graph Construction and Preprocessing, ❷ Process Neighbors Extraction and DFS, ❸ Perturbation and Oversample, ❹ Contrastive Learning and Detection. The pipeline the architecture can be described concisely as follows: The first module takes system logs as inputs and construct a provenance graph based on the inputs. Then the first module applies graph compression technique to obtain a representative provenance graph while preserving attack information, and then initializes node features based on the resulting provenance graph. Subsequently, the second module extracts the neighbor nodes of each process node and conduct a prioritized DFS from these process nodes, from this, we can obtain the initial representation of each process node. For each representation obtained in the previous step, the third module oversamples these vectors that are rarely seen and perturbs them. The last module uses contrastive learning to project the similar vectors and dissimilar vectors to

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 27,2026 at 16:25:56 UTC from IEEE Xplore. Restrictions apply.

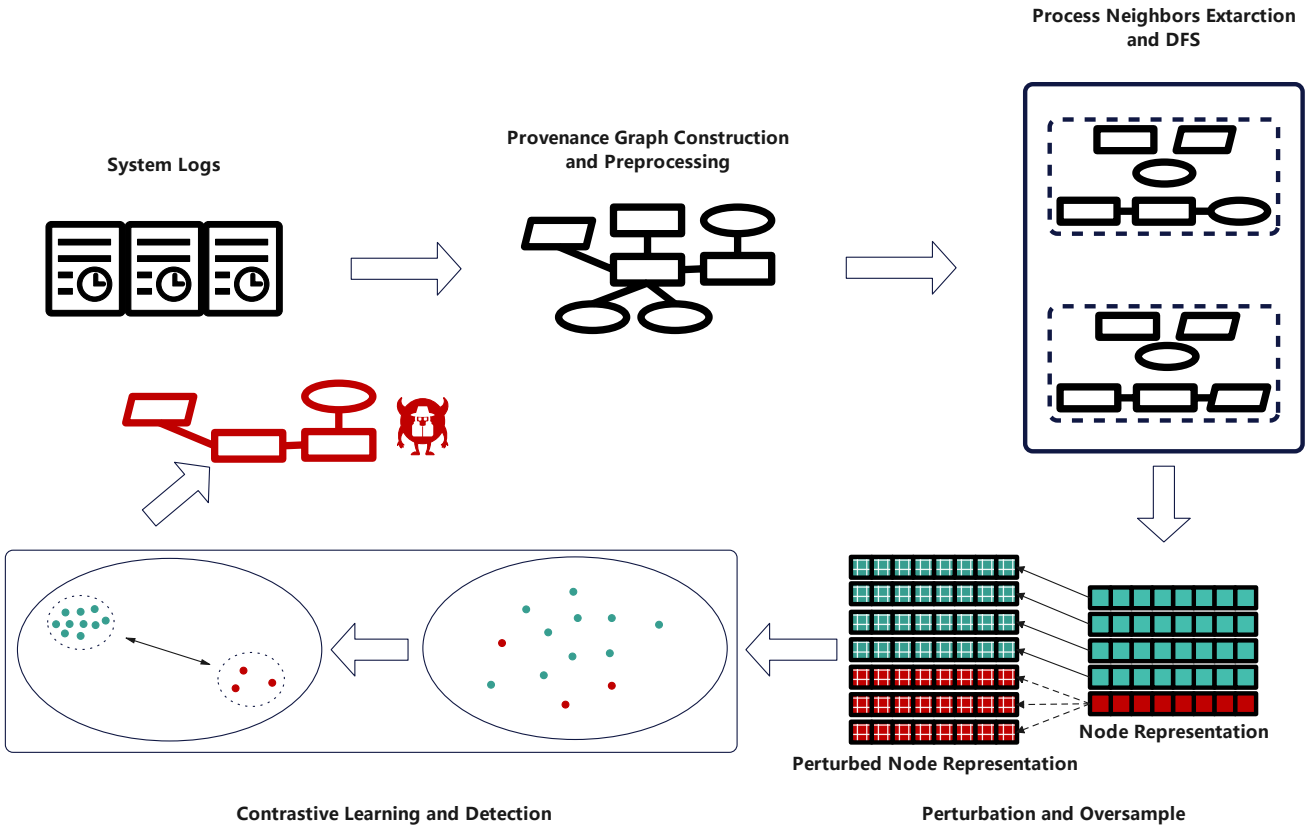


Fig. 2. The architecture of DEEPDIST.

respective space. Finally, using these process entities in attack space, the DEEPDIST can correlate these files and networks they interacted with to recover APT attack campaign.

B. Provenance Graph Construction and Preprocessing

The system logs collected system events, including process events, file events, network events which are the interactions between system entities. Based on these interactions, DEEPDIST can construct a DAG called provenance graph where nodes indicate entities and edges indicate their interaction. By analyzing the provenance graph, DEEPDIST is able to gain valuable insights into the system's behavior, including how different entities interact with each other. However, the provenance graph built from raw system log may be extremely large and involve numerous redundant nodes and edges, this can make it difficult to manipulate in the successive steps. Therefore, it is necessary to compress the graph. we employ graph compression techniques from previous studies [25, 26] to preprocess the provenance graphs. These techniques help to reduce the redundant and attack-irrelevant nodes and edges while retaining the attack information. By compressing the graph, DEEPDIST is able to make the graph more manageable and easier to work with in subsequent analysis steps.

C. Process Neighbors Extraction and DFS

As stated in the motivation, when characterizing nodes, neighbors with an excessively high order can reduce detection granularity (or high false positives), while neighbors with an excessively low order may overlook information relevant to the attack (or high false negatives). In fact, in the scenario of an APT attack, a attack behavior is more like a chain structure than a star structure. Therefore, it is unreasonable to simply consider n-order neighbors. Hence, in DEEPDIST, we not only take into account the n-order neighbors, but also consider the chains deriving from the nodes to be detected.

Take the motivation sample as an example, if we want to detect the `firefox_3148` whether it is an attacking entity, we first extract its all neighbor nodes (e.g., `x.x.x.x`, `explorer.exe_1760`, `z.z.z.z`, `wshbth.dll`, etc.), then adding their one-hot representations. The next step is to extract a chain (or path) from the process node, in this paper, we can adopt the following strategy. Starting from the process node, we perform a depth-first search. Nevertheless, the general depth-first search ignores the relationship between events. For example, a compromised process may fork a child process to execute subsequent tasks rather than operate on files itself. Therefore, we employ a depth-first search with priority. Assuming that the current node is v, among all its neighbor nodes, we first select the

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 27,2026 at 16:25:56 UTC from IEEE Xplore. Restrictions apply.

process node it forks, followed by the file node it writes, because it indicates the data flow direction of v, and finally consider its connected network. In this example, the chain is `firefox_3148`→`plugin-container_2012`→`cmd_744`→`payload.exe_892`. Then we input the sequence of the chain into the LSTM (Long Short-Term Memory) [27], which is utilized to encode the sequence into a vector representation. Finally, the component outputs the concatenation of the added one-hot vectors of all neighbor nodes and the encoding that output by LSTM as the initial representation.

D. Perturbation and Oversample

Due to the significantly smaller number of attack events compared to benign events in the APT attack scenario, employing model for attack detection is hindered by data imbalance. Oversampling is a common method to solve the problem of data imbalance, which balances the dataset by increasing the number of minority class samples. However, simple oversampling increases the risk of overfitting [28], and duplication of samples also results in loss of information. In contrastive learning, data augmentation [20] is a widely used technique to expand datasets, achieved by cropping images, adding noise and other means to generate additional data which can not alter the semantics of the image. However, if we were to apply the same enhancement approach [20, 21] as used in the computer vision field to augment APT data, it would be difficult to ensure that the ground truth of an entity remains unaltered. Therefore, we propose a novel customized data augmentation scheme for APT attacks, which will not change the ground truth of the data. We describe the steps in detail. (1) *Build a benign entity set*: We assume that we can acquire some reliable and known benign entities. The assumption is rational because the benign entities are numerous and ubiquitous in system. In this paper, we assume some benign entities such as the files in the `c:/windows/system32/` directory. We collect these benign entities to construct a benign entity set. (2) *Entity Perturbation*: Entity Perturbation involves randomly adding or removing entities that exist in the benign entity set for each initial node representation obtained from C. The intuition behind this is that the ground truth of an entity is not affected by the addition or removal of its benign interactions, as the ground truth of an entity is determined by whether it involves malicious interactions. In other words, if we add or remove several benign interactions for a benign entity, the entity remains benign. Likewise, if we add or remove several benign interactions for an attack entity, it still remains an attack entity. As a concrete illustration, consider the Firefox process. Typically, this process accesses multiple websites, but it is typically only one site that becomes involved in the attack. Irrespective of any changes made to the benign websites, the compromised state of the Firefox process remains unaffected. By perturbing the original entities by adding or removing benign entity interactions, we obtain more samples. (3) *Oversample*: From the step (1), we have obtained the benign entity set. The entities in the set can be seen as the neighbor nodes of a imaginary node called pseudo-node. Since the neighbors of this pseudo-node are all benign entities, this pseudo-node can also be regarded as a benign entity. We use the same approach as C to encode the pseudo-node, then compare the similarity with the entities to be detected. The entities with low similarity will be oversample.

E. Contrastive Learning and Detection

Now that we have obtained the initial representation vector of each process entity and the oversampled samples, we can proceed to classify them using contrastive learning. Initially, we must project the representations into a relatively low-dimensional space. To accomplish this, we require an encoder to encode these initial representations. In DEEPDIST, we leverage the ResNet (Residual Network) [29] as the encoder to encode these initial representation. The ResNet contains multiple residual blocks with skip connections that allow the input to bypass the fully connected layers and be added to the output. This way, the network can learn residual functions that add to the identity function, making it easier for the network to learn deeper representations. Unlike the original residual network, we substitute convolutional layers with fully connected layers, as our entity representation do not possess local features. We use two linear layers with the same dimensions and activation function for each residual block, with $Relu$ being the chosen activation function. In order to speed up the convergence of the model, the input of each linear layer is processed by batch normalization. The residual block can be denoted as follows:

$$ResBlock(*) = Relu((Relu \circ Liner \circ BN)^2(*) + *) \quad (1)$$

The encoder is simply stacked by multiple residual blocks:

$$Encoder(*) = ResBlock^n(*) \quad (2)$$

Finally we add a projection head on the output of the encoder to project the representation:

$$Y = Tanh \circ Liner \circ Encoder(X) \quad (3)$$

The goal of the projection head is to transform the input data into a feature space where it is easier to distinguish between attack and benign entities. Note that in all equations, $\circ$ and exponential represent composite operation.

Contrastive loss: In order to make the model better distinguish between benign entities and attacking entities, and to cluster the output results of the model into two categories as much as possible (that is, corresponding to benign entities and attacking entities respectively), we need to set a mediation factor α. The mediation factor implicitly guides the model to classify entities as either benign or attack based on the limited number of known benign entities. The calculation of the mediation factor is as follows:

$$\alpha_{i,j} = \begin{cases} 1, & \left|h_i^0 - h_j^0 - h_p^0 \cdot h_i^0 + h_p^0 \cdot h_j^0\right| < 0.2 \\ 0, & otherwise \end{cases} \quad (4)$$

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 27,2026 at 16:25:56 UTC from IEEE Xplore. Restrictions apply.

TABLE I
PROPERTIES OF ADVANCED PERSISTENT THREAT (APT) INCIDENTS IN THE ATLAS DATASET.

Attack ID	APT Campaign	Attack Features[a]						Raw data		Pre-processed data		
		PL	PA	ING	IG	BD	DE	#Entity	#Event	#Entity	#Event	#Attack entity
S-1	Strategic web compromise	√		√	√	√	√	5400	92374	909	3915	11
S-2	Malvertising dominate	√		√	√	√	√	83484	392452	1453	3896	10
S-3	Spam campaign		√	√	√	√	√	6248	124281	1124	6124	18
S-4	Pony campaign		√	√	√	√	√	10072	121986	1163	5704	16
M-1	Strategic web compromise	√		√	√	√	√	11363	243963	2315	10872	20
M-2	Targeted GOV phishing	√		√	√	√	√	17937	277097	2172	10175	27
M-3	Malvertising dominate	√		√	√	√	√	21229	331501	2244	11423	27
M-4	Monero miner by Rig		√	√	√	√	√	11609	254047	2202	11612	23
M-5	Pony campaign		√	√	√	√	√	29655	699765	2292	10690	28
M-6	Spam campaign		√	√	√	√	√	15309	348218	2558	13778	34

[a]PL: Phishing email link. PA : Phishing email attachment. INJ: Injection. IG: Information gathering. BD: Backdoor. DE: Data ex-filtration.

Where the $h_p^0$and h_i^0 denote the initial representation of the pseudo-node and the entity *i* respectively. The contrastive loss function with a mediation factor is as follows:

$$L = -\frac{1}{N}\sum_{i=0}^{N}\log \sum_{j=0,i\neq j}^{N}\alpha_{i,j}\frac{e^{h_i\cdot h_j/\tau}}{e^{h_i\cdot h_j/\tau}+\sum\limits_{k=0,k\neq i,j}^{N}(1-\alpha_{i,k})e^{h_i\cdot h_k/\tau}} \tag{5}$$

The $\cdot$ represents cosine similarity and all operations are broadcastable in (4) and (5).

In our implementation, we project the added one-hot vectors of a process's neighbors to 512 dimensions using a linear layer, and the hidden layer of the LSTM is set to 128 dimensions. Therefore, the dimensions of the ResNet are 640 (512 + 128).

After applying contrastive learning techniques, the model effectively improved the representations of both attack entities and benign entities. Subsequently, we can employ clustering algorithms like DBSCAN [30] to identify the attack process entities. Utilizing these identified entities, we revisit the provenance graph to identify the paths that incorporate these process entities. By merging these paths, we can successfully reconstruct the progress of the APT.

V. EVALUATION

To evaluate the effectiveness of DEEPDIST for detecting APT attacks, we will focus our experiments on the following two research questions:

- **RQ 1**: How effectives is DEEPDIST in APT attack detection? The research question includes the model performance and comparison to baseline.
- **RQ 2**: How does the contrastive learning contribute to DEEPDIST in APT attack detection? Exploring the research question will explain why DEEPDIST can gain better performance in APT detection.

All experiments are performed on a server two Intel Xeon E5-2630 v3 2.4GHz CPUs (32 processors), 128GB memory and NVIDIA Tesla V100 PCIe 32 GB GPU. We implement the DEEPDIST with the PyTorch backend.

TABLE II
THE PROPERTIES OF OUR SIMULATION DATASET.

Scenario ID	#Benign Entity	#Attack Entity	#Benign Event	#Attack Event	#Attack Process
Scenario 1	3355	18	7128	40	3
Scenario 2	3361	35	7129	31	14
Scenario 3	3358	49	7129	50	17
Avg.	3358	34	7128	40	11

A. Dataset

In our evaluation experiment, we adopt the dataset released by ATLAS [13] and our own simulation dataset. The atlas dataset includes ten real-world APT attacks and the audit logs on Windows platform for 24 hours. Out of the ten attacks, four were carried out from a single host machine (S-1 to S-4), while the others came from multiple hosts (M-1 to M-6). Although compression techniques have been adopted for the provenance graph, the percentage of malicious entities remains at only around 1%. The meta attributes of the dataset can be seen in Table I. The simulation dataset contains three attack scenarios and the more information about the simulation is illustrated in Table II.

B. Effectiveness of APT Attack Detection

We applied DEEPDIST to generate two clusters for the entities in Table I and Table II, then we can easily know which one is the cluster of attack entities by the number of entities. We also compared DEEPDIST with two baseline methods, ShadeWatcher and DeepCase [31]. ShadeWatcher has the similar goal to DEEPDIST that find the fine-grained attack characteristics. However, ShadeWatcher requires clean data to train the model, despite claiming robustness to noisy data. DeepCase is a sequence-based attack detection scheme with attention mechanism. The outputted results of DeepCase are also clusters, however, these clusters are not binary which need analyst to inspect them further. As in previous work, we use the classification results, to evaluate the performance of DEEPDIST. Our results show that we can achieve an average precision of 94.91%, recall of 94.74%, and F1-score of 94.69%

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 27,2026 at 16:25:56 UTC from IEEE Xplore. Restrictions apply.

TABLE III
RESULTS OF APT ATTACK ENTITIES DETECTION IN ATLAS.

ID	TP	FP	TN	FN	Precision	Recall	F1-score
S-1	11	0	184	0	100.00%	100.00%	100.00%
S-2	10	2	356	0	83.33%	100.00%	90.91%
S-3	16	0	173	4	100.00%	80.00%	88.89%
S-4	15	0	177	1	100.00%	93.75%	96.77%
M-1	20	0	436	0	100.00%	100.00%	100.00%
M-2	27	0	335	0	100.00%	100.00%	100.00%
M-3	26	1	432	1	96.30%	96.30%	96.30%
M-4	22	2	414	1	91.67%	95.65%	93.62%
M-5	27	1	418	2	96.43%	93.10%	94.74%
M-6	31	5	488	3	86.11%	91.18%	88.57%
Avg.	**20.5**	**1.1**	**341.3**	**1.2**	**94.91%**	**94.47%**	**94.67%**

TABLE IV
RESULTS OF APT ATTACK ENTITIES DETECTION IN SIMULATION DATASET.

ID	TP	FP	TN	FN	Precision	Recall	F1-score
Scenario 1	3	0	1054	0	100%	100%	100%
Scenario 2	14	0	1057	0	100%	100%	100%
Scenario 3	17	0	1061	0	100%	100%	100%
Avg.	11	0	1057	0	100%	100%	100%

in identifying attack entities across the ten attack scenarios in ATLAS dataset and we attained a remarkable accuracy rate of 100% in our simulation dataset . Table III provides a detailed evaluation of DEEPDIST's performance for each scenario in ATLAS dataset and Table IV shows the evaluation results in our simulation dataset. In order to demonstrate the superiority of DEEPDIST, Table V presents the comparative results in comparison to the baselines.

TABLE V
RESULTS OF COMPARISON.

Method	Precision	Recall	F1-score
ShadeWatcher	6.75%	**100%**	12.64%
DeepCase	24.14%	**100%**	38.89%
No Contrast[b]	60.93%	90.29%	71.90%
DEEPDIST	**94.91%**	94.47%	**94.67%**

[b]The data can be obtained from [32].

Case Study. Using the S-2 attack that exploits the `CVE 2015-3105` [33] vulnerability as an illustration, DEEPDIST successfully identified 12 malicious processes during the S-2 attack. Out of these processes, two were false positives (`cmd_3616` and `searchprotocolhost.exe_3816`). However, `searchprotocolhost.exe_3816` was not found to be directly linked to the data flow or control flow of other malicious processes. Fig. 3 depicts the provenance subgraph of this attack, where the malicious processes are highlighted in red rectangles. Consequently, analysts can conveniently conduct attack investigations based on the information provided by DEEPDIST.

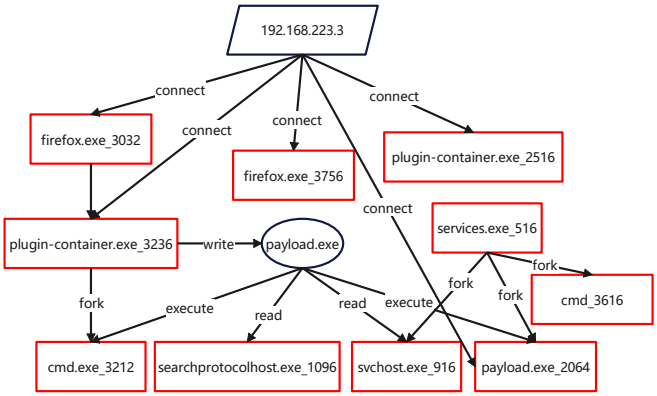


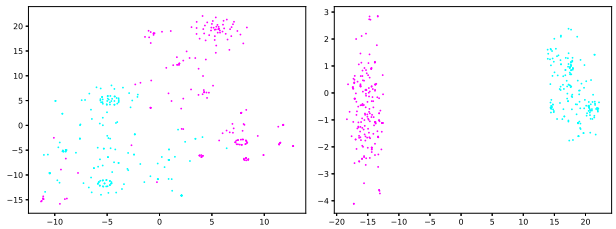
Fig. 3. Case study of Malvertising dominate.

C. The Role of Contrastive Learning

As mentioned in Section II, contrastive learning has the ability to enhance sample representations, thereby improving the model's effectiveness. In this subsection, we will explore how contrastive learning contributes in APT detection.

Table V displays the detection results of the model for APT, both with and without the incorporation of contrastive learning. It is evident from the last two rows that incorporating contrastive learning significantly enhances the model's ability to detect attacks.

In order to explain the function of contrastive learning in attack detection, we take the S-1 attack as an example and employ TSNE to display the feature distribution of nodes with and without the integration of contrastive learning in Fig. 4. Fig. 4a depicts the feature distribution of entities without contrastive learning, while Fig. 4b demonstrates the feature distribution of entities utilizing contrastive learning. It is evident that contrastive learning significantly enhances the ability to distinguish between different entities. Note that the red dots are malicious entities, which we have oversampled for obviousness.



(a) (b)

Fig. 4. Feature distribution with contrastive learning (a) and without contrastive learning (b).

VI. RELATED WORK

Graph-based APT Detection. The majority of current research aimed at detecting APT attacks relies on the utilization

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 27,2026 at 16:25:56 UTC from IEEE Xplore. Restrictions apply.

of provenance graphs. Among these works, some are rule-based detection methods [5, 15]. For example, Holmes [5] leverages the ATT&CK [34] and TTP rules to match the events in time and projects these attack events to a stage of APT attack. There are also methods for APT detection that are based on identifying abnormalities [4, 7, 8, 10, 19]. Typically, these methods involve constructing a model that can predict the normal behavior of the system, and then comparing the system's observed behavior to the model's behavior to identify any deviations as abnormal. To be able to build such models, these efforts usually require large amounts of attack-free data. Furthermore, such methods struggle to discern rare benign behaviors due to the problem of concept drift, thus leading to high false positives. Therefore, NoDoze [23] uses a method based on path and time frequency to score alarms to alleviate the problems of false alarms and alarm fatigue. Not same as these methods, DEEPDIST can identify malicious entities without prior knowledge and attack-free data.

Contrastive Learning. Recently, contrastive learning has achieved great success in the fields of computer vision [20, 21], natural language processing, and graph neural networks [35, 36] as a self-supervised learning technique. It maximizes the similarity between positive samples and minimizes the similarity between negative samples. SimCLR [20] can learn to extract useful features that capture the underlying structure of the images. This allows the network to learn meaningful representations even when labeled data is scarce. In graph learning field, there are also some works that apply contrastive learning to augment node representation. For example, GRACE [35]employs a node-level graph contrastive learning framework that includes corruption techniques like edge removal and node feature masking. D-SLA [36] proposed Discrepancy-based Self-supervised LeArning to ensure the similarity between positive samples. Different from these methods, DEEPDIST customizes a node perturbation scheme for the provenance graph to ensure that the true labels are the same as the original graph.

VII. CONCLUSION

We propose DEEPDIST, a representation-enhanced APT detection using contrastive learning to improve detection effectiveness. In this paper, we propose a customized oversample scheme and entity perturbation method that does not change the ground truth of an entity to counter data imbalance balance. Moreover, we combine the first-order neighbors of the entity and a DFS chain to model an entitiy more accurately. Our evaluation on two datasets shows DEEPDIST can achieve high accuracy in malicious entity detection.

ACKNOWLEDGMENT

This work is supported by the Strategic Priority Research Program of Chinese Academy of Sciences, Grant No.E0HG051105.

REFERENCES

[1] "Advanced persistent threat (APT)," https://www.imperva.com/learn/application-security/apt-advanced-persistent-threat/, 2023.

[2] "CISA Alert: APT Groups Had "Long-Term Access" to a Defense Organization, Exfiltrated Sensitive Data," https://www.cpomagazine.com/cyber-security/cisa-alert-apt-groups-had-long-term-access-to-a-defense-organization-exfiltrated-sensitive-data/, 2023.

[3] "APT Trends Report Q1 2023," https://securelist.com/apt-trends-report-q1-2023/109581/, 2023, Accessed on May 18, 2023.

[4] E. Manzoor, S. M. Milajerdi, and L. Akoglu, "Fast memory-efficient anomaly detection in streaming heterogeneous graphs," in *ACM KDD*, 2016.

[5] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. N. Venkatakrishnan, "HOLMES: Real-time APT detection through correlation of suspicious information flows," in *IEEE Security and Privacy*, 2019.

[6] X. Han, X. Yu, T. F. J. Pasquier, D. Li, J. Rhee, J. W. Mickens, M. I. Seltzer, and H. Chen, "SIGL: Securing software installations through deep graph learning," in *USENIX Security*, 2021.

[7] F. Dong, L. Wang, X. Nie, F. Shao, H. Wang, D. Li, X. Luo, and X. Xiao, "DISTDET: A cost-effective distributed cyber threat detection system," in *USENIX Security*, 2023.

[8] F. Yang, J. Xu, C. Xiong, Z. Li, and K. Zhang, "PROGRAPHER: An anomaly detection system based on provenance graph embedding," in *USENIX Security*, 2023.

[9] S. T. King and P. M. Chen, "Backtracking intrusions," in *SOSP*, 2003.

[10] X. Han, T. F. J. Pasquier, A. Bates, J. Mickens, and M. I. Seltzer, "Unicorn: Runtime provenance-based detector for advanced persistent threats," in *NDSS*, 2020.

[11] F. Liu, Y. Wen, D. Zhang, X. Jiang, X. Xing, and D. Meng, "Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise," in *ACM CCS*, 2019.

[12] H. Irshad, G. F. Ciocarlie, A. Gehani, V. Yegneswaran, K. H. Lee, J. M. Patel, S. Jha, Y. Kwon, D. Xu, and X. Zhang, "TRACE: enterprise-wide provenance tracking for real-time APT detection," *IEEE Transactions on Information Forensics and Security*, 2021.

[13] A. Alsaheel, Y. Nan, S. Ma, L. Yu, G. Walkup, Z. B. Celik, X. Zhang, and D. Xu, "ATLAS: A sequence-based learning approach for attack investigation," in *USENIX Security*, 2021.

[14] P. Fang, P. Gao, C. Liu, E. Ayday, K. Jee, T. Wang, Y. F. Ye, Z. Liu, and X. Xiao, "Back-propagating system dependency impact for attack investigation," in *USENIX Security*, 2022.

[15] W. U. Hassan, A. Bates, and D. Marino, "Tactical provenance analysis for endpoint detection and response systems," in *IEEE Security and Privacy*, 2020.

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 27,2026 at 16:25:56 UTC from IEEE Xplore. Restrictions apply.

[16] K. H. Lee, X. Zhang, and D. Xu, "High accuracy attack provenance via binary-based execution partition," in *NDSS*, 2013.
[17] M. N. Hossain, S. Sheikhi, and R. Sekar, "Combating dependence explosion in forensic analysis using alternative tag propagation semantics," in *IEEE Security and Privacy*, 2020.
[18] W. U. Hassan, M. A. Noureddine, P. Datta, and A. Bates, "OmegaLog: High-fidelity attack investigation via transparent multi-layer log analysis," in *NDSS*, 2020.
[19] J. Zeng, X. Wang, J. Liu, Y. Chen, Z. Liang, T.-S. Chua, and Z. L. Chua, "Shadewatcher: Recommendation-guided cyber threat analysis using system audit records," in *IEEE Security and Privacy*, 2022.
[20] T. Chen, S. Kornblith, M. Norouzi, and G. E. Hinton, "A simple framework for contrastive learning of visual representations," in *ICML*, 2020.
[21] K. He, H. Fan, Y. Wu, S. Xie, and R. B. Girshick, "Momentum contrast for unsupervised visual representation learning," in *CVPR*, 2020.
[22] J. Grill, F. Strub, F. Altché, C. Tallec, P. H. Richemond, E. Buchatskaya, C. Doersch, B. Á. Pires, Z. Guo, M. G. Azar, B. Piot, K. Kavukcuoglu, R. Munos, and M. Valko, "Bootstrap your own latent - A new approach to self-supervised learning," in *NeurIPS*, 2020.
[23] W. U. Hassan, S. Guo, D. Li, Z. Chen, K. Jee, Z. Li, and A. Bates, "Nodoze: Combatting threat alert fatigue with automated provenance triage," in *NDSS*, 2019.
[24] "CVE-2015-5122," https://nvd.nist.gov/vuln/detail/CVE-2015-5122.
[25] Z. Xu, Z. Wu, Z. Li, K. Jee, J. Rhee, X. Xiao, F. Xu, H. Wang, and G. Jiang, "High fidelity data reduction for big data security dependency analyses," in *ACM CCS*, 2016.
[26] M. N. Hossain, J. Wang, R. Sekar, and S. D. Stoller, "Dependence-preserving data compaction for scalable forensic analysis," in *USENIX Security*, 2018.
[27] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, 1997.
[28] G. E. Batista, R. C. Prati, and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," *ACM SIGKDD explorations newsletter*, 2004.
[29] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *CVPR*, 2016.
[30] M. Ester, H. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *ACM KDD*, 1996.
[31] T. van Ede, H. Aghakhani, N. Spahn, R. Bortolameotti, M. Cova, A. Continella, M. van Steen, A. Peter, C. Kruegel, and G. Vigna, "DEEPCASE: Semi-supervised contextual analysis of security events," in *IEEE Security and Privacy*, 2022.
[32] N. Yan, Y. Wen, L. Chen, Y. Wu, B. Zhang, Z. Wang, and D. Meng, "Deepro: Provenance-based APT campaigns detection via GNN," in *TrustCom*, 2022.
[33] "CVE-2015-3105," https://nvd.nist.gov/vuln/detail/CVE-2015-3105.
[34] "MITRE ATT&CK® Matrix for Enterprise," https://securelist.com/apt-trends-report-q1-2023/109581/, 2023, Accessed on May 18, 2023.
[35] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Deep graph contrastive representation learning," in *ICML*, 2020.
[36] D. Kim, J. Baek, and S. J. Hwang, "Graph self-supervised learning with accurate discrepancy learning," in *NeurIPS*, 2022.

Authorized licensed use limited to: University of Education Winneba. Downloaded on July 27,2026 at 16:25:56 UTC from IEEE Xplore. Restrictions apply.